CÓDIGOCLARO · RUTA KOTLIN

Tema 03 · Null safety

Guía de estudio · DAM/DAW

Tipos nullable, ?, ?:, !! y safe calls.

1. Explicación

Un tipo como String no acepta null; String? sí. El operador ?. llama a una propiedad o función solo si el receptor existe, y elvis ?: permite elegir una alternativa. Después de una comprobación clara, el compilador puede aplicar smart cast a un valor no nulo. !! fuerza una conversión y puede lanzar NullPointerException, por lo que no reemplaza el análisis de qué significa la ausencia. Java y otras fronteras pueden introducir tipos plataforma con garantías menores.

2. Ejemplo

Código · kotlin

```kotlin
fun longitud(texto: String?): Int = texto?.length ?: 0

fun main() {
    println(longitud("Kotlin"))
    println(longitud(null))
}
```

Resultado esperado

```
6
0
```

3. Errores frecuentes

- Usar !! para silenciar el compilador sin definir el caso null.
- Olvidar que una expresión safe-call puede devolver null.
- Suponer que la interoperabilidad con Java siempre preserva null safety.

4. Ejercicio

Convierte un texto nullable a entero. Devuelve -1 si es null o si no representa un entero.

Casos que debe superar

Entrada	Resultado esperado
"12"	12
"x"	-1
null	-1

5. Solución explicada

Código · kotlin

```kotlin
fun convertir(texto: String?): Int = texto?.toIntOrNull() ?: -1
```

Criterio de aceptación: Se acepta una comprobación explícita con if si evita !! y respeta el valor alternativo especificado.

Otras soluciones válidas

- Usar if (texto == null) para tratar la ausencia.
- Usar safe-call con Elvis como en la solución orientativa.

6. Fuentes oficiales

JetBrains / Kotlin — Null safety